WireProof: Supplementary Reproduction Notes

Hoang Nhat Khang Nguyen Minh Hieu Le Huynh-Gia-Huy Pham
Phat T. Tran-Truong Xuan-Bach Le

6 November 2026

Supplement to *WireProof: Verifiable Post-Quantum Cryptography Migration for Service Ecosystems*. This document describes the preserved local feasibility demonstration, its evidence, and its current limits. Planned mesh studies and formal verification remain future work.

1 Artifact and environment

The artifact is at `https://github.com/lexuanbach/wireproof-artifact`. The measured code and evidence were first packaged at commit `4dca0715cc8f4563847c4b1b5f3d7078470a16d4`. The supplementary documentation does not change that prototype or its measurements. The measured environment was Go 1.26.1, Python 3.15.2, and macOS/arm64. The prototype uses standard libraries and local loopback connections. Reproduction requires Go 1.26 or later and Python 3; no root access, Docker, or external service is required.

2 Reproduction

From the repository root, run:

```
bash prototype/reproduce.sh runs/reproduced-01
```

The destination is relative to `prototype/` and must not already exist. The script runs the Go race-enabled tests, Go vet, and Python parser tests; builds the demonstration; executes six scenarios ten times each; replays the recorded schedules; checks captures independently; and generates summaries. Fresh execution creates new test certificates, timestamps, and hashes. Do not overwrite preserved runs. The root `REPRODUCE.md` separately gives the pinned commands for the prior QuantumTrustKG results; those results are not new WireProof experiments.

For inspection of the measured run without opening fresh TLS connections:

```
cd prototype
go build -o wireproof-demo .
./wireproof-demo replay -dir runs/final-03
python3 oracle.py runs/final-03
./wireproof-demo summarize -dir runs/final-03
```

Replay recomputes decisions from recorded events; it does not rerun the network experiment. Use a copy of the checkout if regenerated derived files must be kept separate from the original snapshot.

3 Preserved results

The paper reports only `prototype/runs/final-03`. Development and extracted-package validation runs remain separate and their counts are not pooled. The six fixed scenarios have ten trials each:

Scenario	Checks passed	Connections	Strict rejections
Hybrid	50/50	10	10
Fallback	20/20	10	0
Rollout	60/60	20	0
Delay	60/60	20	0
Later violation	40/40	20	0
Reassignment	90/90	10	0
Total	320/320	90	10

All 60 measured trials completed without a trial error. The Python oracle checks 100 captures: 90 successful negotiated-group agreements and ten expected strict-rejection cases. For successful connections it parses the ServerHello group independently of endpoint telemetry. Rejection classification also depends on the endpoint error and absence of a ServerHello. This is not a complete authentication-algorithm oracle.

The later-violation case deliberately delays reports by 50 ms. The interval from the later connection response to first discovery ranges from 50.155 to 51.156 ms, with median 51.135 ms. Both ledgers become pending at discovery. This local discovery interval does not establish a latency advantage or invalidate the earlier historical population. Ten repetitions of fixed scenarios do not justify the planned bound based on 59 independent noncompliant trials.

4 Evidence navigation and integrity

- `source-sha256.json` records the nine source files, measured environment, and executable digest. `evidence-sha256.json` records 850 measured evidence entries. The packaged executable and ephemeral private test keys are excluded; their recorded digests are retained.
- `manifest.json`, `launch.json`, and server logs record process bindings, endpoints, public observer keys, and negotiation telemetry.
- Trial directories retain `events.jsonl`, `trial.json`, `replay.json`, raw TCP captures, and socket mappings. These connect signed observations, appraisal history, current-eligibility changes, and actual checks.
- `summary.json`, `summary.csv`, `summary.tex`, and `oracle.json` hold the observed counts and independent capture checks.
- The root `CONTRACT-SKETCH.md`, `EVALUATION.md`, and `PROTOCOL-WP2.md` describe intended semantics and planned studies. Their requirements are not claims of completed implementation.

5 Current limits and unresolved cases

The harness trusts process launch, control events, clock, registry completeness, and endpoint keys. Operation attribution uses request metadata and executable/configuration hashes. This establishes neither hardware attestation nor Kubernetes workload discovery. The baseline receives the same events and shares reported-population guards and history; complete-population requests unsupported by the baseline are treated as a capability gap.

Four additional event sequences remain unresolved in this snapshot:

1. An invalidation received before its assignment can be forgotten.
2. A wrong-policy assessment can revoke eligibility, followed by confirmation under the same epoch.
3. Conflicting authenticated payloads reusing an observation identifier can leave the original fact usable.
4. A late historical correction does not itself withdraw current eligibility before another assessment.

The recorded fixed-scenario successes therefore establish feasibility rather than correctness under every asynchronous schedule. These four cases need fixes and new validation before stronger lifecycle claims. No prototype source or recorded result was changed to prepare this supplement.

General rolling windows, autonomous expiry, full authentication-algorithm observation, distributed completeness, Kubernetes/Istio integration, receipt-signature experiments, and new mechanized proofs remain research targets. The planned 24-month programme evaluates accuracy, availability, coverage, and cost under matched claims and independently established labels.